

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 5:

Study and Empirical Analysis of Prim and Kruskal Greedy Algorithms

Elaborated:

st. gr. FAF-241

Pleșu Dinu

Verified:

asist. univ.

Fiștic Cristofor

TABLE OF CONTENTS

INTRODUCTION	3
1 ALGORITHM ANALYSIS	4
1.1 Objective	4
1.2 Tasks	4
1.3 Theoretical Notes	4
1.4 Comparison Metric	5
1.5 Input Format	6
2 IMPLEMENTATION	7
2.1 Prim	7
2.2 Kruskal	9
2.3 Comparative Analysis	11
CONCLUSION	13
REFERENCES	14

INTRODUCTION

A minimum spanning tree of a connected weighted undirected graph is a subgraph that connects all vertices using exactly $|V| - 1$ edges such that the total edge weight is minimized. Minimum spanning trees arise naturally in a wide range of practical problems: designing minimum-cost telecommunication or electrical networks, constructing road or pipeline infrastructure that connects all sites with the least total length, clustering data points by removing the heaviest edges from the spanning tree, and approximating solutions to the Travelling Salesman Problem. The MST of a graph is not necessarily unique, but its total weight is always the same regardless of which specific MST is selected.

Two classical algorithms for constructing minimum spanning trees are studied in this laboratory. Prim's algorithm grows the spanning tree vertex by vertex starting from an arbitrary source. At each step the cheapest edge connecting a vertex already in the tree to a vertex not yet included is added, until all vertices belong to the tree. Kruskal's algorithm takes a global view: it sorts all edges in non-decreasing order of weight and adds each edge to the forest if and only if it does not create a cycle, continuing until $|V| - 1$ edges have been selected.

Both algorithms are guaranteed to produce a minimum spanning tree on any connected weighted undirected graph, but they do so through fundamentally different strategies. Prim resembles Dijkstra's algorithm in structure and is well-suited to dense graphs where many edges are available at each step. Kruskal is better suited to sparse graphs where sorting the edges is cheap relative to the total work. In practice, the choice between them depends on the graph structure and the quality of the available data structures.

The efficiency of both algorithms depends critically on the internal data structures used. For Prim, the bottleneck is the selection of the next cheapest edge; using a priority queue instead of a linear scan reduces this from $O(V^2)$ to $O(E \log V)$ total. For Kruskal, the bottleneck is the cycle-detection step; using a union-find data structure with path compression and union by rank reduces this to nearly $O(E \alpha(V))$ amortized, where α is the inverse Ackermann function, which is effectively constant for all practical graph sizes.

Classic and optimized variants of both algorithms were tested on connected weighted undirected graphs at several sizes and density levels. The aim was to see how much the optimized data structures actually help in practice and whether the choice between Prim and Kruskal matters more than density level.

1 ALGORITHM ANALYSIS

1.1 Objective

Study and analyze Prim and Kruskal on connected weighted undirected graphs with increasing size and different density.

1.2 Tasks

1. study the greedy algorithm design technique;
2. implement Prim and Kruskal in a programming language;
3. perform empirical analysis of the two algorithms;
4. increase the number of nodes in the graph and analyze the influence on the algorithms;
5. make a graphical presentation of the obtained data;
6. formulate conclusions.

1.3 Theoretical Notes

Greedy algorithms build a solution incrementally by always making the locally optimal choice at each step, without reconsidering earlier decisions. For a greedy strategy to yield a globally optimal solution, the problem must exhibit the greedy-choice property: a global optimum can always be reached by including the locally best choice made at the current step. The minimum spanning tree problem possesses this property, which can be formalized through matroid theory. The graph matroid structure guarantees that any greedy algorithm that always selects the minimum-weight edge not forming a cycle will produce a minimum spanning tree [1, 2].

The correctness of MST algorithms rests on two fundamental properties of minimum spanning trees. The cut property states that for any cut of the graph (a partition of vertices into two non-empty sets), the minimum-weight edge crossing the cut belongs to some minimum spanning tree. Prim exploits this property directly: at each step the growing tree forms one side of a cut, and the cheapest crossing edge is added. The cycle property states that for any cycle in the graph, the maximum-weight edge of that cycle does not belong to any minimum spanning tree (assuming unique edge weights). Kruskal exploits this property: it never adds an edge that closes a cycle, so the heaviest edge in any cycle is always excluded.

Prim's algorithm in its classic form maintains a key array recording the minimum-weight edge from each vertex to the current tree. At each of the V steps, the algorithm scans all unselected vertices to find the one with the smallest key, costing $O(V)$ per step and $O(V^2)$ in total. For each newly added vertex, the key values of its neighbors are updated if a cheaper connection is found, adding $O(E)$ total update work.

The overall classic complexity is therefore $O(V^2 + E) = O(V^2)$. The optimized variant stores unselected vertices in a binary min-heap keyed by their current minimum edge weight. Each extraction costs $O(\log V)$ and each key update (decrease-key) costs $O(\log V)$, giving $O(V \log V)$ for all extractions and $O(E \log V)$ for all updates, for an overall complexity of $O(E \log V)$. On dense graphs with $E = O(V^2)$, the classic $O(V^2)$ can be faster than the optimized $O(V^2 \log V)$ due to lower constant factors [3].

Kruskal’s algorithm first sorts all $|E|$ edges by weight, costing $O(E \log E)$. It then processes each edge in sorted order and checks whether its two endpoints belong to different components of the current forest. The classic implementation represents each component as a set of vertices; the component membership test requires a linear scan taking $O(V)$ in the worst case, giving $O(V \cdot E)$ for all tests. The optimized implementation uses a union-find data structure with path compression and union by rank. The `find` operation with path compression identifies the component root in nearly $O(1)$ amortized time, and the `union` operation merges two components in $O(\alpha(V))$ amortized time, where α is the inverse Ackermann function. The total complexity is therefore $O(E \log E + E \cdot \alpha(V)) = O(E \log E)$ dominated by the sorting step.

The practical choice between Prim and Kruskal depends on graph density. On dense graphs where $E = O(V^2)$, the edge list for Kruskal contains $O(V^2)$ elements and sorting costs $O(V^2 \log V)$, while Prim’s classic variant costs only $O(V^2)$. Dense graphs therefore favor Prim. On sparse graphs where $E = O(V)$, Kruskal’s sorting costs $O(V \log V)$ and its union-find operations cost nearly $O(V)$, while Prim’s heap variant costs $O(E \log V) = O(V \log V)$ — the two are comparable, but Kruskal often wins due to its simpler access pattern and better cache locality when processing a sorted edge array sequentially.

1.4 Comparison Metric

Four values were recorded for each run:

- execution time in milliseconds;
- peak memory usage in kilobytes;
- total weight of the minimum spanning tree;
- number of edges and graph density.

1.5 Input Format

The benchmark used connected weighted undirected graphs with random edge weights from 1 to 20. Three graph categories were generated:

- sparse graphs;
- medium-density graphs;
- dense graphs.

The tested graph sizes were 100, 200, 350, 500, and 700 vertices.

2 IMPLEMENTATION

2.1 Prim

Two Prim variants were implemented. The classic version selects the next vertex by scanning all candidates linearly; the optimized one uses a priority queue instead, bringing the per-step cost down from $O(V)$ to $O(\log V)$ [4].

2.1.1 Algorithm Description

The method follows the idea shown in the next pseudocode.

```
Prim(graph, start):
    mark start as part of the tree
    push all outgoing edges of start in priority queue
    while the queue is not empty:
        extract the cheapest edge
        if it reaches an unvisited vertex:
            add that edge to the tree
            add outgoing edges of the new vertex to the queue
    return minimum spanning tree
```

2.1.2 Implementation

The heap brings a clear speed advantage, but every newly added vertex pushes its neighbors into the queue, so dense graphs drive memory usage up considerably.

```
# Classic --  $O(V^2)$  linear minimum selection over adjacency matrix
def prim_classic(graph, start=0):
    n = graph.num_vertices
    matrix = graph.to_matrix()
    selected = [False] * n
    min_edge = [float("inf")] * n
    min_edge[start] = 0
    total_weight = 0
    for _ in range(n):
        v = min((i for i in range(n) if not selected[i]),
                key=lambda i: min_edge[i])
        selected[v] = True
        total_weight += min_edge[v]
        for u in range(n):
            w = matrix[v][u]
            if w and not selected[u] and w < min_edge[u]:
                min_edge[u] = w
    return total_weight

# Optimized --  $O(E \log V)$  heap-based priority queue
def prim_optimized(graph, start=0):
    visited = set()
    heap = [(0, start)]
    total_weight = 0
    while heap and len(visited) < graph.num_vertices:
        weight, vertex = heapq.heappop(heap)
        if vertex in visited:
            continue
        visited.add(vertex)
        total_weight += weight
        for neighbor, edge_weight in graph.adj[vertex]:
            if neighbor not in visited:
                heapq.heappush(heap, (edge_weight, neighbor))
    return total_weight
```

2.1.3 Results

Table 2.1 shows the results at the largest dense graph (700 nodes).

Table 2.1 – MST benchmark on dense graphs at 700 nodes

Algorithm	Median time (ms)	Median memory (KB)	MST weight	Edges
Kruskal_Classic	589.288	1582.46	699	122760.0
Kruskal_Optimized	545.990	1440.25	699	122760.0
Prim_Classic	1330.551	3879.47	699	122760.0
Prim_Optimized	435.591	7302.00	699	122760.0

The optimized Prim variant was fastest at 435.6 ms, but also consumed the most memory — the time-space trade-off of the heap is directly visible in the data. Figure 2.1 shows overall performance across all sizes.

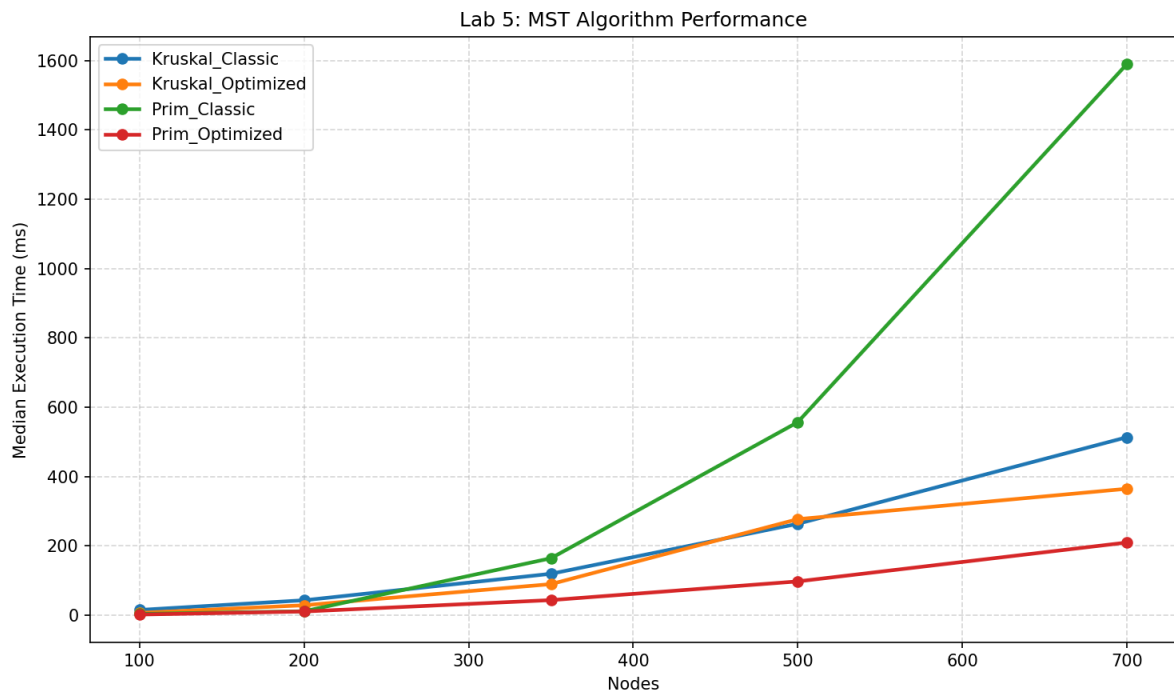


Figure 2.1 – Overall MST performance

The growth is especially pronounced for classic Prim, which struggles even on moderately sized graphs.

2.1.4 Conclusion

Prim is effective for minimum spanning tree construction, but its practical performance depends heavily on the data structure used for edge selection. At 700 nodes on dense graphs the optimized variant required 435.6 ms while the classic required 1330.6 ms, a speedup of approximately $3.1\times$. On sparse graphs the advantage was even larger: the optimized variant required only 126.3 ms while the classic required 1318.2 ms, a speedup of approximately $10.4\times$, because the heap contains far fewer candidate

edges on sparse graphs and each extraction is correspondingly cheaper. The trade-off is memory: on the 700-node dense graph the optimized variant consumed 7302 KB due to heap allocation, while the classic required only 3879 KB.

2.2 Kruskal

Two Kruskal variants were also implemented. The classic one detects cycles by scanning component sets linearly; the optimized one uses union-find with path compression and rank, making each check nearly constant-time.

2.2.1 Algorithm Description

The Kruskal method follows the idea shown in the next pseudocode.

```
Kruskal(graph):
    sort all edges increasingly by weight
    make one component for each vertex
    for each edge in sorted order:
        if the two endpoints are in different components:
            add edge to the tree
            merge the two components
    return minimum spanning tree
```

2.2.2 Implementation

The union-find structure is compact — two integer arrays of length V — so the optimized variant also tends to use less memory than the set-of-sets approach of the classic version.

```
# Classic --  $O(E^2)$  linear component search
def kruskal_classic(graph):
    components = [{v} for v in range(graph.num_vertices)]
    total_weight = 0
    for edge in sorted(graph.edges):
        li = next(i for i, c in enumerate(components) if edge.source in c)
        ri = next(i for i, c in enumerate(components) if edge.target in c)
        if li == ri:
            continue
        total_weight += edge.weight
        components[li] |= components[ri]
        components.pop(ri)
    return total_weight

# Optimized --  $O(E \log E)$  with union-find (path compression + rank)
def kruskal_optimized(graph):
    parent = list(range(graph.num_vertices))
    rank = [0] * graph.num_vertices
    def find(v):
        while parent[v] != v:
            parent[v] = parent[parent[v]]
            v = parent[v]
        return v
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra; rank[ra] += 1
        return True
    total_weight = 0
    for edge in sorted(graph.edges):
        if union(edge.source, edge.target):
```

```

        total_weight += edge.weight
    return total_weight

```

2.2.3 Results

Table 2.2 shows the Kruskal results at 700 sparse nodes.

Table 2.2 – MST benchmark on sparse graphs at 700 nodes

Algorithm	Median time (ms)	Median memory (KB)	MST weight	Edges
Kruskal_Classic	362.709	458.16	715	25219.0
Kruskal_Optimized	227.098	315.95	715	25219.0
Prim_Classic	1318.228	3879.47	715	25219.0
Prim_Optimized	126.301	1421.54	715	25219.0

The optimized variant was faster (227.1 ms vs 362.7 ms) and also used less memory. Figure 2.2 breaks the results down by density.

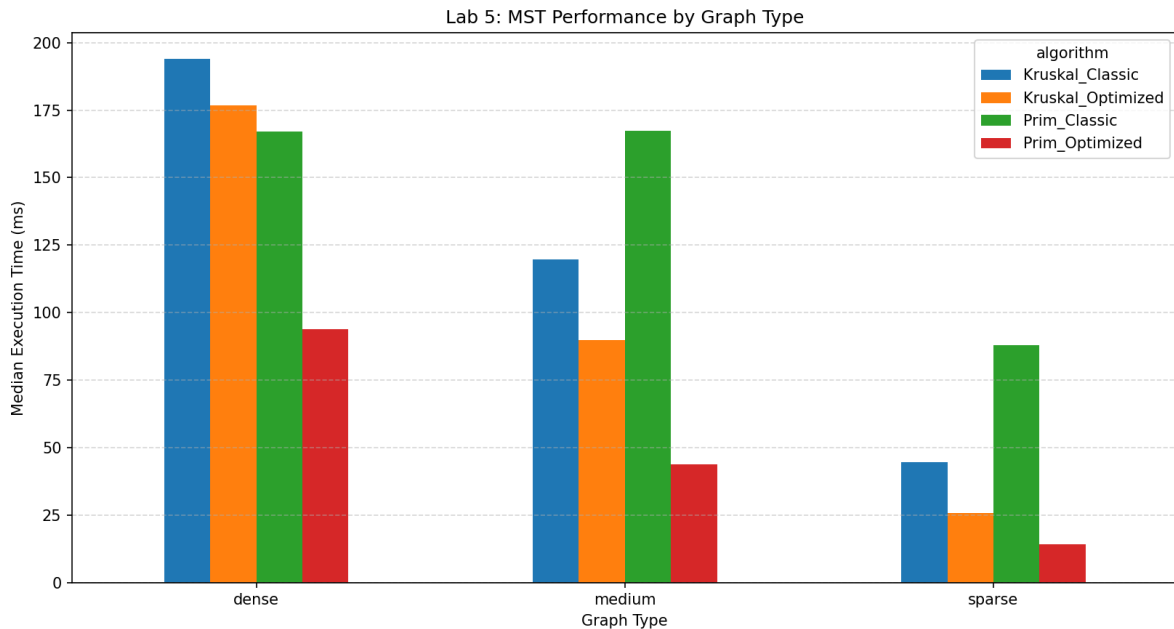


Figure 2.2 – MST performance by graph type

As Figure 2.2 shows, density affects all four variants — more edges means more work for every implementation.

2.2.4 Conclusion

Kruskal remains a strong greedy solution for MST construction, especially when combined with an efficient union-find structure. On sparse graphs the optimized variant showed a clear benefit: at 700 sparse nodes it required 227.1 ms against the classic's 362.7 ms, a speedup of $1.6\times$. On dense graphs the advantage was smaller because the large edge list makes the sorting step dominant for both variants: at 700 dense nodes the optimized required 546.0 ms against the classic's 589.3 ms. In both cases the optimized

variant also consumed less memory, owing to the compact union-find arrays compared with the set-of-sets representation of the classic implementation.

2.3 Comparative Analysis

Both algorithms produced the same MST weight on every test case, confirming their correctness. Where they differ is in speed, memory, and how sensitive they are to graph density.

Prim optimized was the fastest variant overall, averaging a $4.47\times$ speedup over Prim classic. The classic variant came last on every input: its $O(V)$ linear scan costs the same regardless of how many edges are present, which wastes time on sparse graphs. Kruskal optimized was consistently faster than Kruskal classic, with the largest difference on sparse graphs where sorting is cheap and the union-find savings dominate.

Memory is where the two algorithms differ most clearly. Figure 2.3 shows peak consumption across all tested sizes. Prim’s heap enqueues all incident edges of each newly added vertex, and on dense graphs this accumulates fast — 7302 KB at 700 dense nodes. Kruskal’s union-find stores only two arrays of length V , so its footprint barely changes with edge count: 1440 KB at the same point.

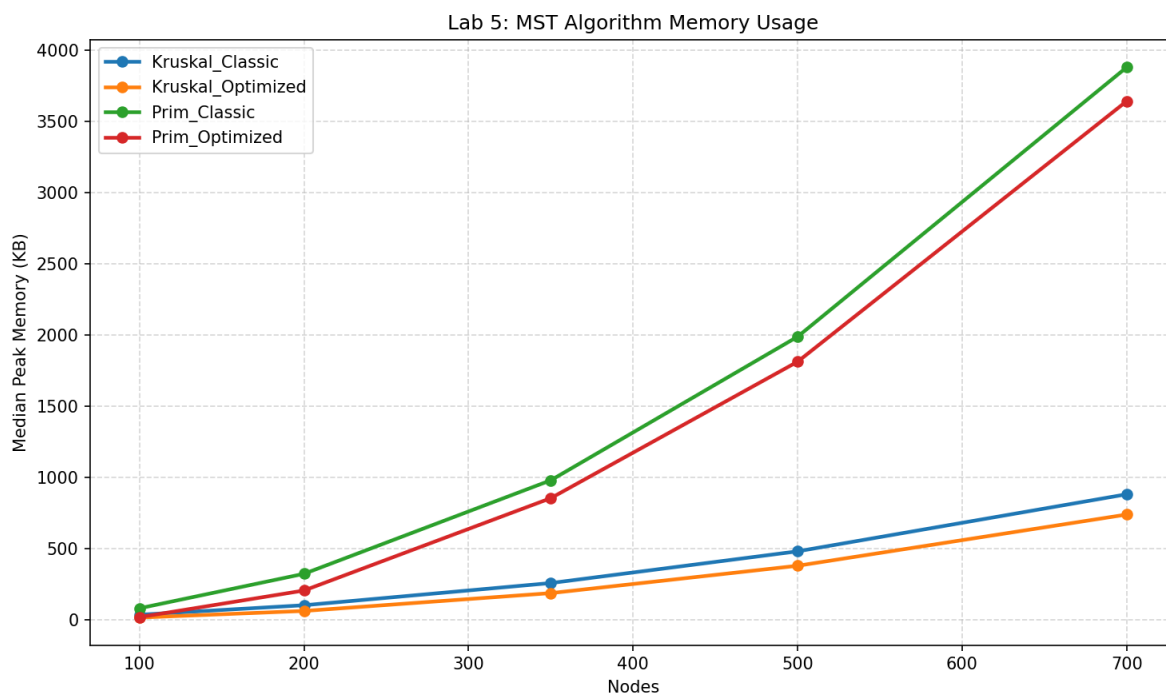


Figure 2.3 – Memory usage of MST algorithms across all graph sizes

Figure 2.4 breaks memory usage down by density. Prim’s consumption rises sharply from sparse to dense; Kruskal’s stays nearly flat because the union-find size depends only on vertex count. In a memory-limited setting, Kruskal with union-find is the safer choice even when Prim optimized would be faster.

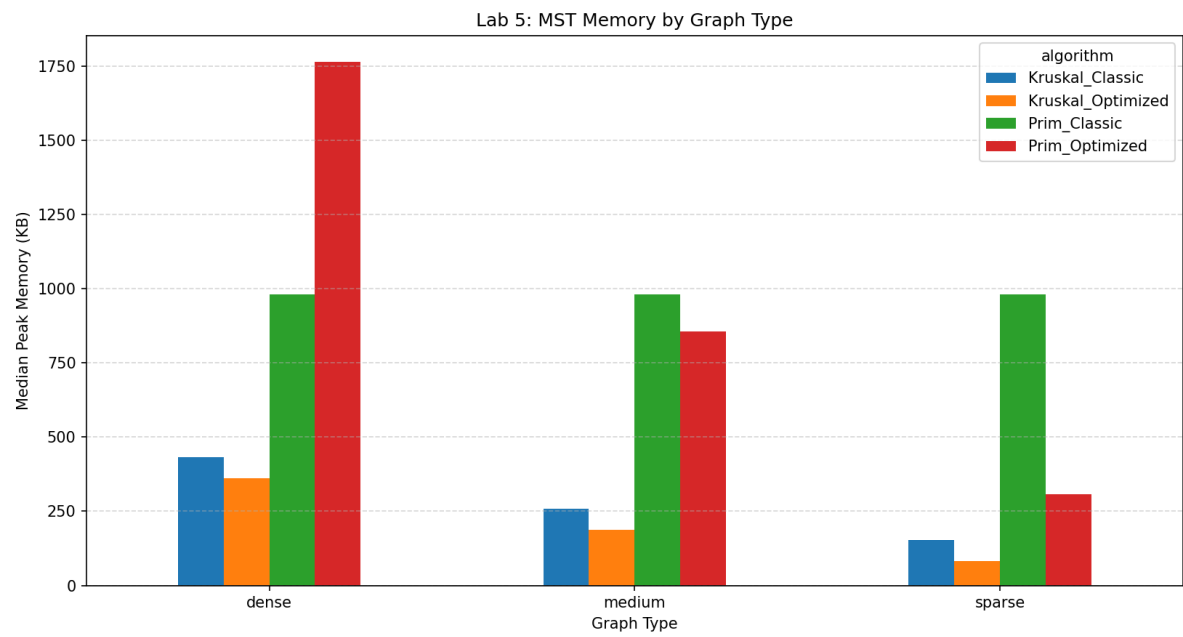


Figure 2.4 – Memory usage of MST algorithms by graph type

CONCLUSION

Both algorithms slowed down noticeably on denser and larger graphs, but the optimized data structures made a significant difference in both cases. Prim optimized was the overall winner, averaging a $4.47\times$ speedup over Prim classic across the full benchmark.

Among the tested variants, Prim classic performed worst on all inputs because its $O(V^2)$ linear scan is equally expensive regardless of edge density. The optimized Kruskal implementation improved on the classic Kruskal by $1.6\times$ on sparse graphs but offered only modest gains on dense graphs, where the dominant cost is sorting the large edge list. Prim optimized was the fastest on both sparse and dense graphs, but at the cost of substantially higher memory usage on dense inputs.

The memory difference at 700 dense nodes — 7302 KB for Prim optimized versus 1440 KB for Kruskal optimized — makes the practical choice straightforward: if memory is not a concern, Prim optimized is faster; if it is, Kruskal with union-find is the safer option. Either way, the optimized implementations should be preferred over the classic ones at all tested density levels.

REFERENCES

- [1] GeeksforGeeks. *Greedy Algorithms* [online]. [cited 07.05.2026]. Available: <https://www.geeksforgeeks.org/greedy-algorithms/>.
- [2] TutorialsPoint. *Greedy Algorithms* [online]. [cited 07.05.2026]. Available: https://www.tutorialspoint.com/data_structures_algorithms/greedy_algorithms.htm.
- [3] Scaler. *Prim's and Kruskal's Algorithm* [online]. [cited 07.05.2026]. Available: <https://www.scaler.com/topics/prims-and-kruskal-algorithm/>.
- [4] Pleşu D. *aa-course-repo, Lab 5 source folder* [online]. GitHub, 2026. [cited 07.05.2026]. Available: <https://github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab5>.